

```
1. daily_ohlc = (142.56, 145.32, 142.08, 143.78)
2.
3. # Access individual values
4. open_price = daily_ohlc[0] # 142.56
5. close_price = daily_ohlc[3] # 143.78
6.
7. # Slicing works too
8. high_low = daily_ohlc[1:3] # (145.32, 142.08)
9.
```

Tuples also support negative indexing and slicing, just like lists.

Tuple Immutability: Preventing Costly Mistakes

The key characteristic of tuples is that they cannot be modified after creation. Attempting to change a tuple will result in an error:

```
1. daily_ohlc = (142.56, 145.32, 142.08, 143.78)
2.
3. # This would cause an error
4. # daily_ohlc[3] = 144.00 # TypeError: 'tuple' object does not support item assignment
5.
```

This immutability is a feature, not a limitation. It provides data integrity for critical information that should not be accidentally modified.

*?? **Did You Know?** In 2012, Knight Capital Group lost \$440 million in just 45 minutes due to a software error that sent erroneous orders to the market. One contributing factor was improper management of trading parameters that should have remained constant. Today, many trading firms use immutable data structures like tuples to store critical order parameters and risk limits precisely to prevent this type of catastrophic error. By making certain values immutable, they ensure that core trading parameters cannot be accidentally modified during execution.*

Tuple Unpacking: Extracting Components Elegantly

Tuples have a special feature called "unpacking" that makes working with fixed-format data particularly convenient: